

File/Directory Control & Local Variables

Operation	Keystroke	Function	Note
Emacs Variables	Emacs supports specifying Emacs variable values. This can be quite useful to control Emacs behaviour for specified file, file types or files or file types inside a specific location. - For example you can: - identify the tab-width for a specific file, or - the imenu-generic-expression pattern for a specific type of file, extending what is already available to match the DSL you are using inside a specific directory. The variables can be specified using 3 methods: Method 1: File Variables. Written inside the file to which the variables apply, using file variables written as special comment at the top or at the bottom of the file. Method 2: Per-Directory Local Variables. Written inside a special Emacs-specific file (.dir-local.el) to control the value of variables inside all or specified files in the directory or directory tree. Method 3: With Emacs Lisp code that identifies class of variables for specific file types, files in a directory tree. See: Using Directory Local Variable classes for individual files @ Emacs Wiki dir-locals-set-class-variables and dir-locals-set-directory-class @ Emacs Manual in Per-Directory Local Variables - This method will be required if you want to use Emacs file or directory variables for a project where those are not accepted. You can put the required code inside your init.el file. Currently PEL does not support configuring it using customization.		
Last updated on:	2026-09-19		
Open this PDF file. See also: 📖 Help/Info	<f11> f v <f1>	(pel-help-pdf &optional OPEN-WEB-PAGE)	Open the 📖 File/Directory Variables local PDF. With C-u , M-- , M- -1 or C- -1 prefix: open the remote GitHub hosted PDF instead. If the pel-flip-help-pdf-arg user-option is set it's the other way around.
Inside a file: File Variables See also: Local variables File Local Variables	File variables control Emacs variable for the file into which they are specified. Emacs supports 2 ways to write the file variable specification inside a file, these commands support the first of those two ways: 1: the specification of the variable written inside the very first line using the following format: <pre> -*- mode: modename; var: value; ... -*- ;; -*- mode: Lisp; fill-column: 75; comment-column: 50; -*- </pre> - For example, inside a Lisp file: 2: at the end of a file with a line containing the string ‘Local Variables:’, and ends with a line containing the string ‘End:’. - For example, in a C file: <pre> /* Local Variables: */ /* mode: c */ /* comment-column: 0 */ /* End: */ </pre> The following commands can be used to add, modify and remove file variable specification on the first line of a file. Notes🔧: - The file-local-variables-alist variable holds an association list of file variable names to their value. - For some major modes PEL supports customization of important behaviour control variables, such as variable controlling indentation like c-basic-offset for cc-modes. For those, the values selected by PEL customization are overridden the values set via file variables.		
Add file local variable at end of file	<f11> f v l	(add-file-local-variable VARIABLE VALUE &optional INTERACTIVE)	Add file-local VARIABLE with its VALUE to the Local Variables list. - This command deletes all existing settings of VARIABLE (except ‘mode’ and ‘eval’) and adds a new file-local VARIABLE with VALUE to the Local Variables list. - If there is no Local Variables list in the current file buffer, then this function adds it at the end of the file, with the first line containing the string ‘Local Variables:’ and the last line containing the string ‘End:’.
Add a file variable on first line	<f11> f v =	(add-file-local-variable-prop-line VARIABLE VALUE &optional INTERACTIVE)	Add file-local VARIABLE with its VALUE to the -*- line. - This command prompts for variable name and value. - It deletes all existing settings of VARIABLE (except ‘mode’ and ‘eval’) and adds a new file-local VARIABLE with VALUE to the -*- line. - If there is no -*- line at the beginning of the current file buffer then this function adds it.
Delete a file variable from first line	<f11> f v -	(delete-file-local-variable-prop-line VARIABLE &optional INTERACTIVE)	Delete all settings of file-local VARIABLE from the -*- line.
Copy all directory file variables to first line	<f11> f v c	(copy-dir-locals-to-file-locals-prop-line)	Copy directory-local variables to the -*- line.
Per-Directory Local Variables dir-local format Safety of File Variables See also: Directory variables	Emacs supports directory local variables that can apply to all files inside a given directory and its sub-directory tree. - The variables can be restricted by major modes and sub-directories. - These files are stored inside a file named .dir-locals.el stored in the directory it controls. - File local variables override directory local variables, so main settings may be stored inside the directory and can then be fined-tuned inside files. - Normally directory local variables are stored inside a file named .dir-locals.el. It can also be stored inside the file .dir-locals-2.el. Emacs loads them both, and loads the -2 file after the other. That way you can put the .dir-locals.el under version control and use the other to make personal or temporary customizations. The following commands can be used to create and modify the content of the .dir-locals.el file located in the current directory. 🙌 If you are familiar with Emacs Lisp it is probably easier to edit the .dir-locals.el file directly, following the format described in the Emacs Manual Directory Local Variables section and to be aware of the security concerns and the ability to force customization of settings that may be identified as unsafe by reading the Safety of File Variables section. You may want to customize the value of safe-local-variable-values to allow some of your values. In any case, it's best to check the code in .dir-locals.el and to check the value of safe-local-variable-values. ⚠ Mode Hook Problem: Directory variables are loaded after the major mode and its hooks are executed, preventing mode hooks to see the variables. 🖥 Because Emacs normal-mode function calls (set-auto-mode) first and later calls (hack-local-variables) if at all. Two techniques exists to circumvent this problem, and PEL uses them to allow controlling important variables like imenu-generic-expression that controls the parsing to extract imenu items.		
Add a directory variable	<f11> f v D =	(add-dir-local-variable MODE VARIABLE VALUE)	Add directory-local VARIABLE with its VALUE and MODE to .dir-locals.el.
Delete a directory variable	<f11> f v D -	(delete-dir-local-variable MODE VARIABLE)	Delete all MODE settings of file-local VARIABLE from .dir-locals.el.
Copy file local variable to directory .dir-locals.el	<f11> f v D C	(copy-file-locals-to-dir-locals)	Copy file-local variables to .dir-locals.el.
Others	The following commands can be used to see and control the current directory.		
List/change buffer's current default directory	M-x cd RET	(cd DIR)	Prompt: shows current directory and allows to change the buffer's default directory to a new DIRectory.
	- If your environment includes a ‘CDPATH’ variable, try each one of that list of directories (separated by occurrences of ‘path-separator’) when resolving a relative directory name. - The path separator is colon in GNU and GNU-like systems.		
Change current working directory back to directory holding visited file	<f11> M-.	(pel-cd-to-current &optional SILENT)	Change current directory to the directory holding visited file, if any. Print message showing the new current working directory if it changed unless SILENT is non-nil (can be requested by prefix argument).

File & Directory Local Variables — References

Topic & Link	Description
GNU Emacs Manual: File Variables	Describes the concept of Emacs file variables
GNU Emacs: Safety of File Variables	Describes the concept of Emacs directory local variables
GNU Emacs: Per-Directory Local Variables	Describes the safety issues related to Emacs file variables and Emacs way of dealing with safety.
Local Variables @ Emacs Wiki	Short and sweet overview of the concept with description of the interaction with mode hooks. Must read for Emacs Lisp dev.